



CS117

PHÂN RÃ VẤN ĐỀ - DECOMPOSITION

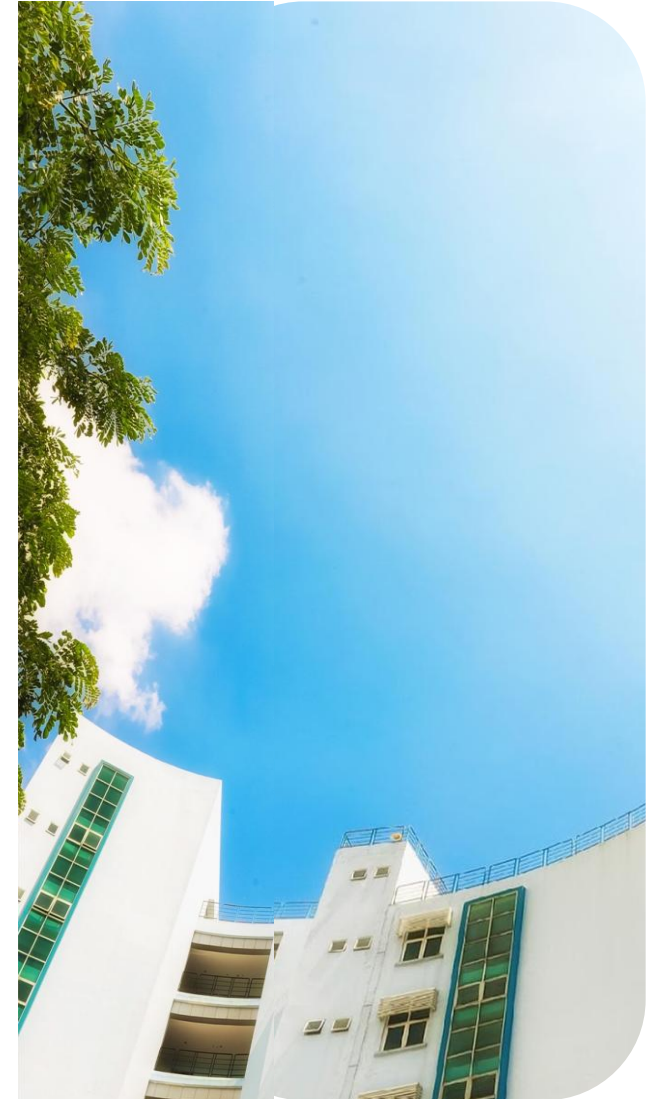
Giảng viên: NGÔ ĐỨC THÀNH



KHÁI NIỆM DECOMPOSITION

Phân rã bài toán (Decomposition) là như thế nào ?

- **Decomposition** là quá trình chia một vấn đề phức tạp thành nhiều phần nhỏ hơn **để dễ giải quyết hơn**.
 - Thay vì giải quyết toàn bộ bài toán lớn, ta phân tách nó thành các bài toán con nhỏ hơn, dễ kiểm soát hơn.
 - Sau đó, giải từng phần rồi kết hợp lại để có lời giải hoàn chỉnh
- Ví dụ,
 - Khi lập kế hoạch tổ chức một sự kiện, chúng ta tách ra các phần: đặt địa điểm, mời khách, chuẩn bị thực phẩm, thiết kế chương trình...





Tại sao Decomposition quan trọng?

- **Lợi ích chính của Decomposition:**
 1. **Giảm độ phức tạp** → Bài toán lớn trở nên đơn giản hơn khi chia nhỏ.
 2. **Dễ quản lý hơn** → Nhóm phát triển có thể phân công xử lý từng phần.
 3. **Tăng khả năng tái sử dụng** → Các phần nhỏ có thể được sử dụng lại cho bài toán khác.
 4. **Dễ dàng gỡ lỗi** → Nếu có lỗi, chỉ cần kiểm tra từng phần nhỏ thay vì toàn bộ hệ thống.
- Ví dụ với lập trình: Khi viết một chương trình lớn, ta chia thành các hàm, module nhỏ để dễ xử lý.



Decomposition trong Computational Thinking

- Computational Thinking (CT) gồm 4 thành phần chính:
 - Decomposition (Phân rã vấn đề).
 - Pattern Recognition (Nhận dạng mẫu).
 - Abstraction (Trừu tượng hóa).
 - Algorithm Design (Thiết kế thuật toán).
- Vai trò của Decomposition trong CT:
 - Thông thường, là bước đầu tiên khi giải quyết vấn đề bằng máy tính.
 - Nếu không phân rã vấn đề tốt, các bước như nhận dạng mẫu và thiết kế thuật toán sẽ khó thực hiện.



Các cách tiếp cận Decomposition

- Có nhiều cách để phân rã một vấn đề, tùy thuộc vào đặc điểm của bài toán và mục tiêu giải quyết.
- Ba cấp độ decomposition chính trong khoa học máy tính và lập trình:
 - Decomposition theo chức năng (Functional Decomposition).
 - Decomposition theo dữ liệu (Data Decomposition).
 - Decomposition theo quy trình (Process Decomposition).
- Mỗi cấp độ có cách tiếp cận khác nhau và áp dụng phù hợp trong các bài toán thực tế.



Các cách tiếp cận Decomposition

1. Decomposition theo chức năng (Functional Decomposition)

- ✓ Chia bài toán thành các chức năng hoặc module con, mỗi module đảm nhận một nhiệm vụ cụ thể.
- ✓ Ứng dụng: Lập trình phần mềm, kiến trúc hệ thống.

2. Decomposition theo dữ liệu (Data Decomposition)

- ✓ Chia dữ liệu thành các phần nhỏ hơn để xử lý song song hoặc độc lập.
- ✓ Ứng dụng: xử lý dữ liệu lớn, lập trình đa luồng, phân tán dữ liệu.

3. Decomposition theo quy trình (Process Decomposition)

- ✓ Chia vấn đề thành các bước hoặc giai đoạn xử lý tuần tự.
- ✓ Ứng dụng: Lập trình hướng quy trình (Process-Oriented Programming), thiết kế thuật toán.

Các cách tiếp cận Decomposition

Tiêu chí	Decomposition theo chức năng	Decomposition theo dữ liệu	Decomposition theo quy trình
Cách tiếp cận	Chia bài toán theo các chức năng, module	Chia dữ liệu thành các phần nhỏ hơn để xử lý riêng	Chia thành các bước hoặc giai đoạn xử lý
Ứng dụng chính	Thiết kế phần mềm, Kiến trúc hệ thống	Xử lý dữ liệu lớn, Lập trình song song	Xây dựng thuật toán, Tự động hóa quy trình
Ví dụ thực tế	Hệ thống đặt hàng: Tách thành Đặt hàng, Thanh toán, Gửi thông báo	Xử lý dữ liệu thời tiết: Chia theo vùng địa lý, xử lý song song	Quản lý quy trình sản xuất: Nhập liệu → Xử lý → Báo cáo
Ưu điểm	Dễ quản lý, dễ mở rộng	Tăng tốc độ xử lý, tận dụng tài nguyên tốt hơn	Giúp tổ chức công việc một cách logic, giảm sai sót
Nhược điểm	Nếu không tổ chức tốt có thể dẫn đến dư thừa code	Cần kỹ thuật tối ưu hóa để tránh dư thừa dữ liệu	Không phù hợp cho các bài toán yêu cầu xử lý song song



ỨNG DỤNG DECOMPOSITION



Decomposition trong Lập trình

Trong lập trình, Decomposition giúp tổ chức code tốt hơn, tăng khả năng bảo trì và tái sử dụng.

Các cách sử dụng decomposition trong lập trình:

- **Chia chương trình thành các hàm (Function Decomposition):** chia một chương trình lớn thành các hàm nhỏ, mỗi hàm thực hiện một nhiệm vụ cụ thể.
- **Sử dụng lập trình hướng đối tượng (Object-Oriented Decomposition):** sử dụng lớp (class) và đối tượng (object) để tổ chức chương trình theo mô hình hướng đối tượng.
- **Chia nhỏ dự án thành các module (Modular Programming):** Chia chương trình lớn thành các module độc lập, có thể được nhập vào khi cần thiết.



Decomposition trong Thuật toán

Trong thiết kế thuật toán, Decomposition giúp chia bài toán lớn thành các bài toán con nhỏ hơn, giúp:

- **Tăng hiệu suất:** Chia nhỏ bài toán giúp tối ưu hóa bộ nhớ và thời gian thực thi.
- **Dễ kiểm tra và sửa lỗi:** Mỗi phần có thể được kiểm thử riêng biệt.
- **Tận dụng lập trình song song:** Một số bài toán có thể chia nhỏ để xử lý đồng thời.

Một số kỹ thuật decomposition phổ biến trong thuật toán:

- Divide and Conquer (Chia để trị) – Ứng dụng trong Merge Sort, Quick Sort.
- Dynamic Programming (Quy hoạch động) – Ứng dụng trong Fibonacci, Knapsack.
- Graph Decomposition – Ứng dụng trong tìm đường ngắn nhất, hệ thống mạng.
- ...



Áp Dụng Decomposition vào Problem Solving

Bước 1: Xác định vấn đề lớn

- Đầu tiên, hiểu rõ vấn đề cần giải quyết.
- Đặt câu hỏi 5W1H (What, Why, Who, When, Where, How), 5 Whys để nắm rõ bản chất.

Bước 2: Phân rã vấn đề thành các phần nhỏ hơn

- Tìm những thành phần chính của vấn đề.
- Cách tiếp cận decomposition:
 - Chia theo chức năng (Functional Decomposition)
 - Chia theo dữ liệu (Data Decomposition)
 - Chia theo quy trình (Process Decomposition)

Bước 3: Giải quyết vấn đề từng phần.

Bước 4: Tổng hợp giải pháp và đánh giá kết quả

- Sau khi giải quyết từng phần, tổng hợp để kiểm tra đã được khắc phục chưa.
- Nếu chưa, lặp lại quá trình decomposition để đi sâu hơn vào nguyên nhân gốc rễ.



Ví dụ về Decomposition trong Problem Solving

💡 Bước 1: Xác định vấn đề lớn

- Người dùng phản hồi rằng website tải chậm, ảnh hưởng đến trải nghiệm.

💡 Bước 2: Phân rã vấn đề

- Máy chủ (Server) chậm >> Cần nâng cấp hoặc tối ưu.
- Code front-end không tối ưu >> Có quá nhiều file CSS/JS tải không cần thiết.
- Cơ sở dữ liệu truy vấn chậm >> Cần tối ưu bảng dữ liệu, cache.
- Hình ảnh quá lớn >> Nén ảnh để giảm dung lượng.

💡 Bước 3: Giải quyết từng phần

- Nhóm backend kiểm tra máy chủ và database.
- Nhóm frontend tối ưu mã nguồn, giảm số lượng request HTTP.
- Nhóm thiết kế nén ảnh mà không làm giảm chất lượng.

💡 Bước 4: Tổng hợp & kiểm tra

- Sau khi tối ưu từng phần, kiểm tra lại tốc độ website để đảm bảo vấn đề được giải quyết hoàn toàn.



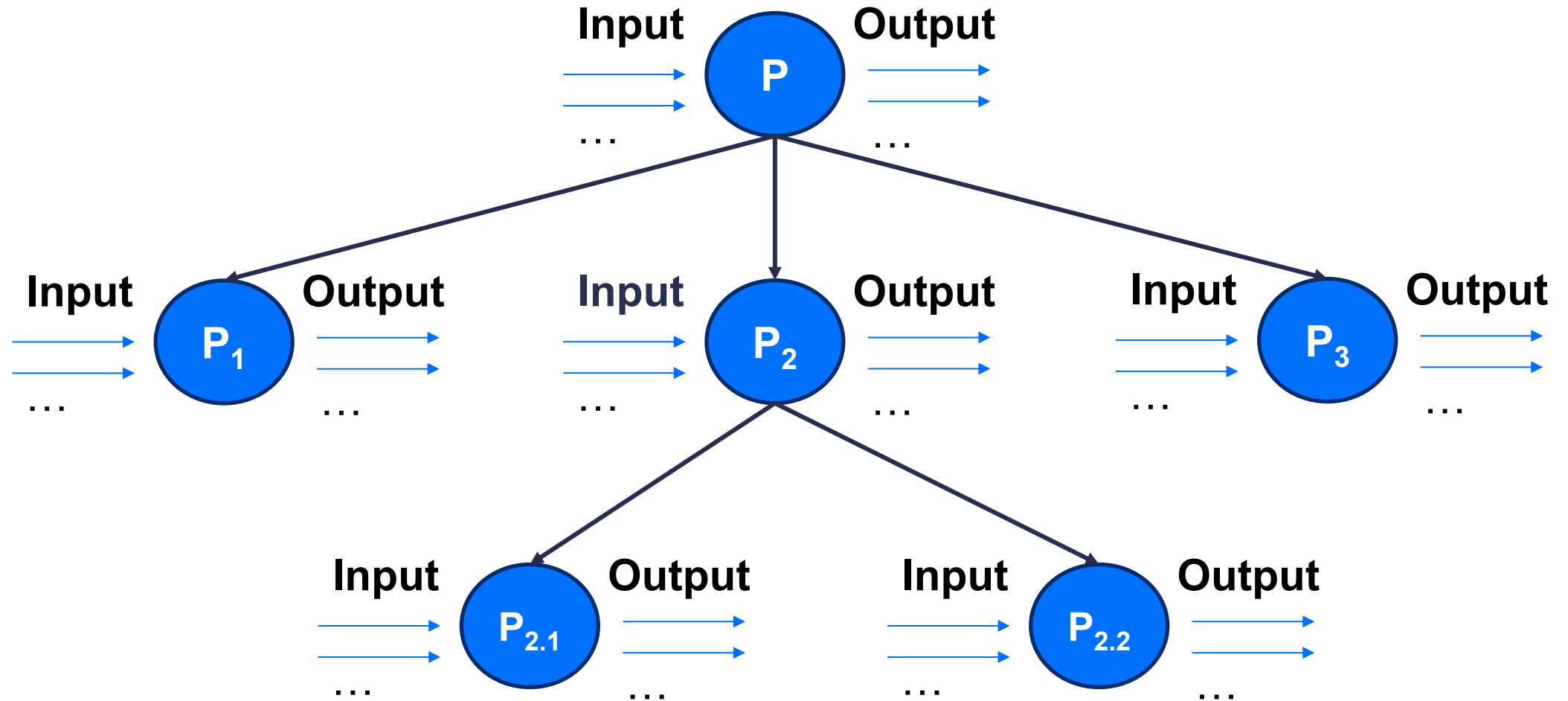
BIỂU DIỄN VÀ PHÂN TÍCH KẾT QUẢ DECOMPOSITION

Biểu diễn kết quả Decomposition

Biểu diễn trực quan hóa kết quả decomposition là một bước thiết yếu trong tư duy giải quyết vấn đề có hệ thống, đóng vai trò như một công cụ trung gian giữa phân tích tư duy và thiết kế giải pháp cụ thể.

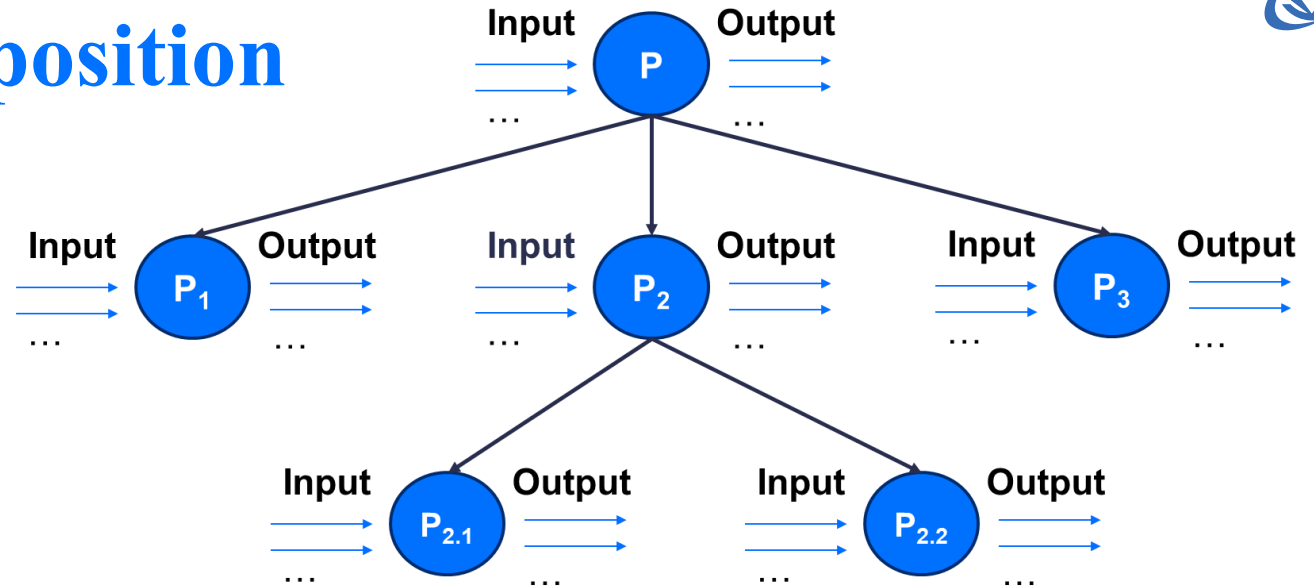
- **Phương pháp Breakdown Tree** – Biểu diễn phân cấp có cấu trúc là một hình thức biểu diễn trực quan theo cấu trúc cây (tree structure), mô hình hóa mối quan hệ giữa bài toán chính và các bài toán con phát sinh từ quá trình phân rã.
 - **Nút gốc (root node):** đại diện cho bài toán tổng thể (overall problem).
 - **Các nút trung gian (intermediate nodes):** biểu thị các nhóm chức năng hoặc bài toán con (sub-problems/modules).
 - **Các nút lá (leaf nodes):** biểu diễn các bài toán hay tác vụ cụ thể, có thể được xử lý độc lập hoặc triển khai bằng thuật toán, mô hình, hoặc công cụ phần mềm.

Biểu diễn kết quả Decomposition





Biểu diễn kết quả Decomposition



- **Mỗi node trên cây là một problem.**
- Giải pháp của problem ở node gốc, được tổng hợp dựa trên giải pháp của các problem ở tất cả các node lá.
- Độ sâu của cây tương ứng với **khả năng làm chủ giải pháp tổng thể.**
- Input/Output của từng node có **liên hệ chặt chẽ về logic** với Input/Output của các node khác trên cây.
- Cây phân rã không thể hiện flow.

Khi nào dừng phân rã ?

Decomposition là công cụ hỗ trợ tư duy, không phải là mục tiêu cuối cùng. Việc dừng decomposition đúng lúc giúp tập trung vào giải pháp thay vì quá tập trung vào phân tích.

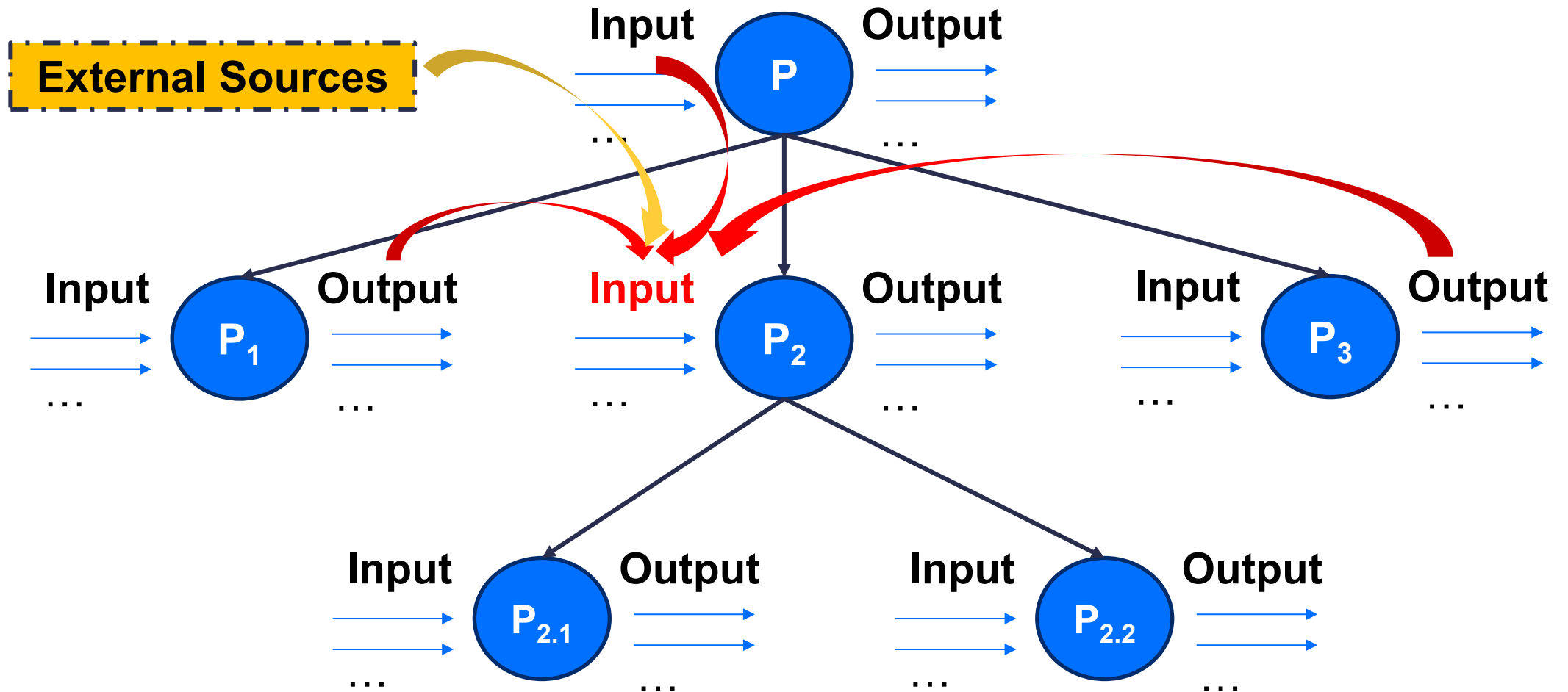
- **Dừng phân rã khi:**

- ✓ Các bài toán con đã đủ cụ thể và rõ ràng để giải quyết riêng biệt.
- ✓ Mỗi bài toán có mục tiêu rõ ràng và có thể đưa ra giải pháp mà không cần hiểu toàn bộ bài toán ban đầu.
- ✓ Có thể giao nhiệm vụ cho từng bài toán hoặc nhóm xử lý mà không bị phụ thuộc lẫn nhau quá mức.
- ✓ Khi chi phí phân tích sâu hơn vượt quá lợi ích đạt được – ví dụ: mất nhiều thời gian nhưng không làm giải pháp tốt hơn.

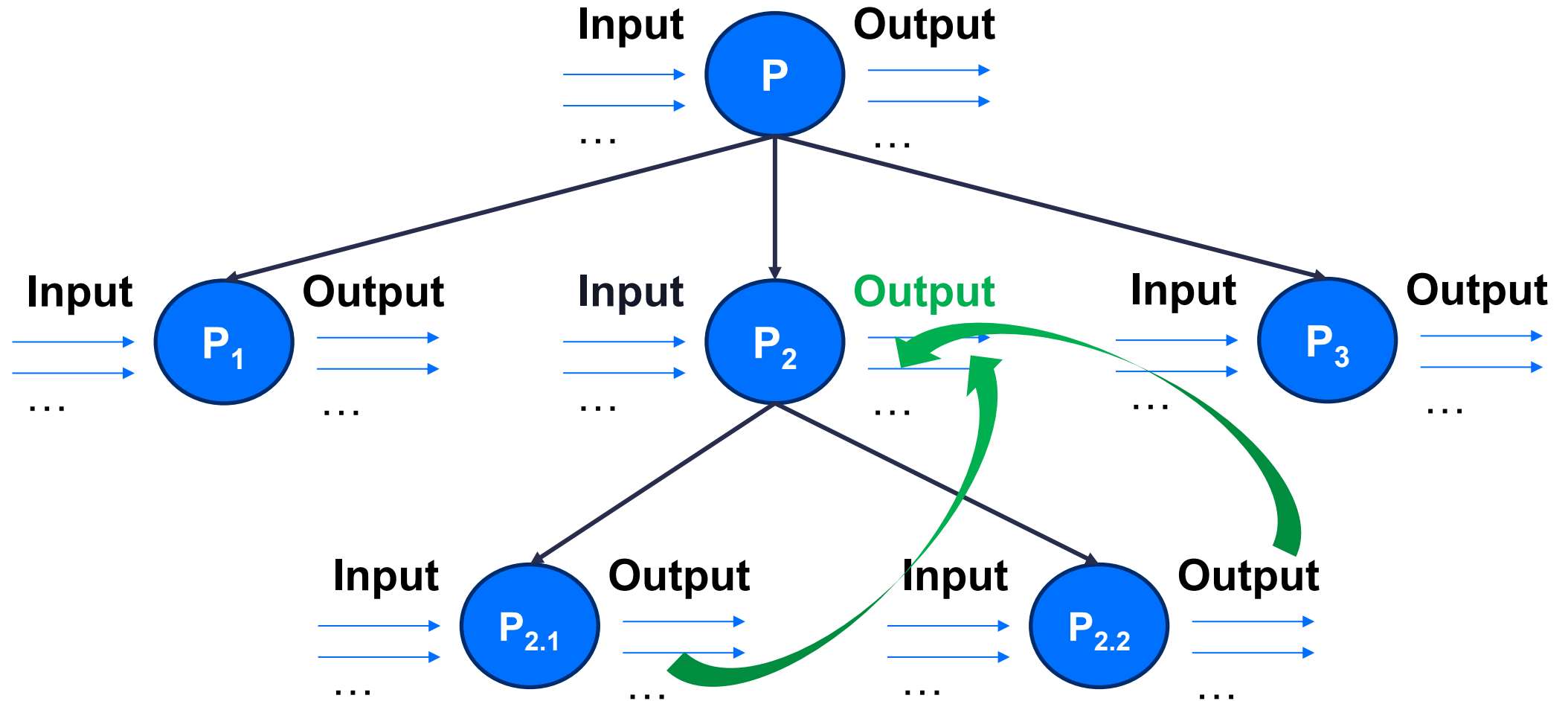


LOGIC TRÊN BREAKDOWN TREE

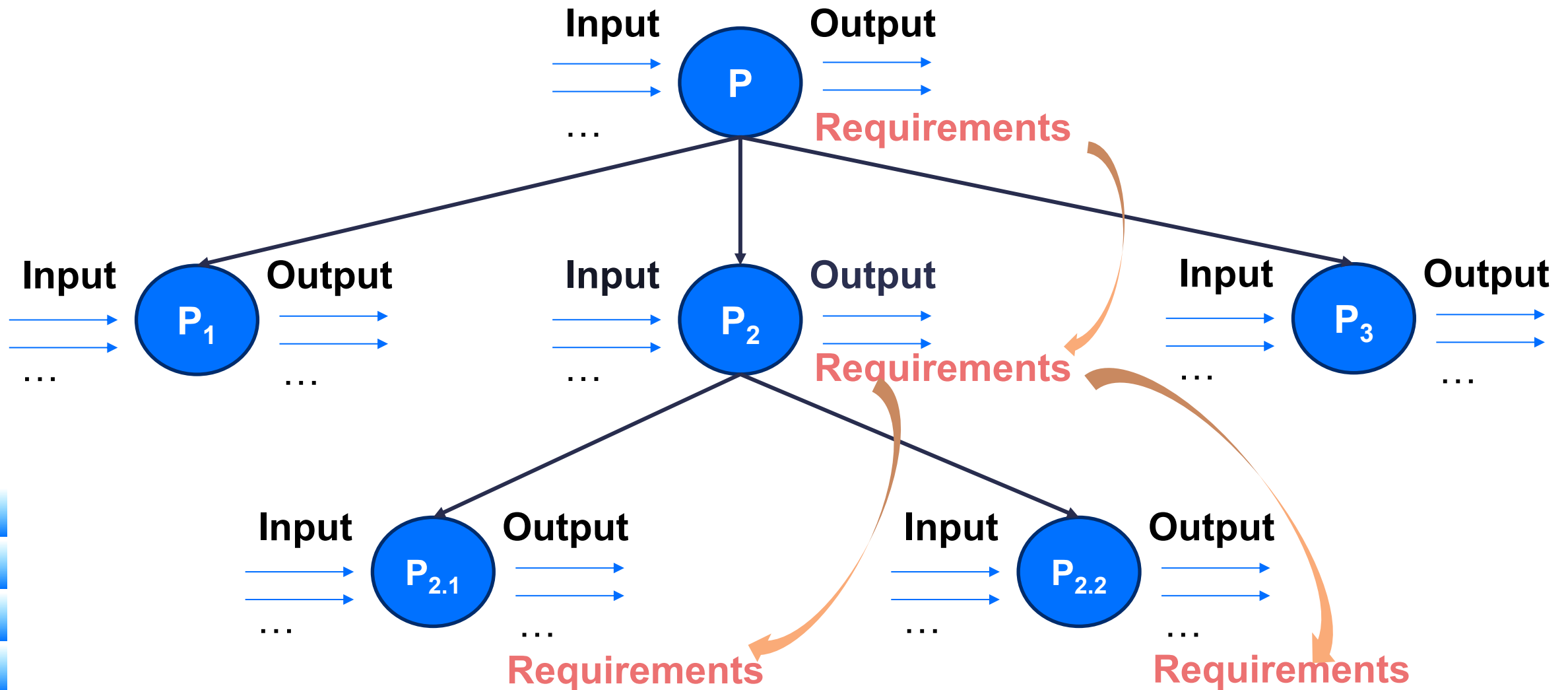
Phân tích kết quả Decomposition



Phân tích kết quả Decomposition



Phân tích kết quả Decomposition





Logic trong quy trình giải quyết vấn đề

Decomposition là bước khởi đầu, nhưng không hoạt động độc lập. Để giải quyết vấn đề hiệu quả, cần kết hợp nó với các kỹ năng khác của Tư duy Tính toán.

- 1. Decomposition + Pattern Recognition** → sau khi phân rã, ta nhận diện các phần giống nhau hoặc có cấu trúc lặp lại để tìm quy luật.
- 2. Decomposition + Abstraction** → từ các phần nhỏ, ta trừu tượng hóa những chi tiết không quan trọng để tập trung vào bản chất.
- 3. Decomposition + Algorithmic Thinking** → mỗi phần sau khi phân rã cần có thuật toán xử lý cụ thể.
- 4. Decomposition + Evaluation** → đánh giá hiệu quả từng phần và cả giải pháp tổng thể sau khi đã ghép lại.



CHÂN THÀNH CẢM ƠN